

OData Endpoint Guide

A comprehensive reference for developers and integration engineers working with SAP Gateway OData services. Master query options, filtering, pagination, and troubleshooting.

 [Query Options](#) [Best Practices](#) [\\$metadata](#) [Query Builder](#)

Overview

OData (Open Data Protocol) is a standardized REST-based protocol for building and consuming queryable APIs. In the SAP ecosystem, OData powers SAP Gateway, enabling external applications to interact with SAP business data and processes.

→ RESTful Design

Uses standard HTTP methods (GET, POST, PUT, DELETE) and follows REST conventions.

→ Query Capable

Rich query options for filtering, sorting, paging, and selecting specific fields.

Self-Describing

The \$metadata endpoint provides complete schema information for the service.

OData Guide

services.

- **Interoperability:** Works with any HTTP client—no SAP-specific libraries required.
- **Fiori Integration:** SAP Fiori apps are built on OData, making it the native API layer.
- **Tooling Support:** Extensive code generation tools for various languages and frameworks.

↪ Endpoint Anatomy

Understanding the structure of an OData URL is fundamental. Every endpoint follows a predictable pattern that makes navigation intuitive once you know the components.

URL Structure

General Pattern

`https://<host>:<port>/sap/opu/odata/sap/<SERVICE_NAME>/<EntitySet>(<Key>)?<Quer`

Example breakdown:

<code>https://my-s4hana.com:443/sap/opu/odata/sap/API_SALES_ORDER_SRV/A_SalesOrder</code>
— Protocol + Host — — Gateway Path — — Service — — Entity Set —



Service Root

The base URL up to and including the service name. This is your starting point.

`https://host/sap/opu/odata/sap/API_SALES_ORDER_SRV`

Entity Set

A collection of entities (like a database table). Appended to the service root.

`.../API_SALES_ORDER_SRV/A_SalesOrder`

OData Guide

```
.../A_SalesOrder('0000000123')
```

Common Examples

Fetching All Sales Orders

```
GET /sap/opu/odata/sap/API_SALES_ORDER_SRV/A_SalesOrder
```

Fetching a Specific Sales Order

```
GET /sap/opu/odata/sap/API_SALES_ORDER_SRV/A_SalesOrder('0000000123')
```

Navigating to Related Items

```
GET /sap/opu/odata/sap/API_SALES_ORDER_SRV/A_SalesOrder('0000000123')/to_Item
```



Navigation Properties

Use navigation properties (like `to_Item`) to traverse relationships. Check `$metadata` to discover available navigations for each entity.



\$metadata

The `$metadata` endpoint is your API blueprint. It describes the complete schema of the service—entity types, properties, relationships, and capabilities.

Accessing Metadata

```
GET /sap/opu/odata/sap/API_SALES_ORDER_SRV/$metadata
```

What's Inside

EntityType

Edit with  Lovable

OData Guide

EntitySet

A named collection of EntityType instances. This is what you query in URLs.

NavigationProperty

Defines relationships between entities (e.g., SalesOrder → SalesOrderItem).

FunctionImport

Custom operations beyond CRUD (e.g., "ApproveOrder", "CalculatePrice").

Sample Metadata Snippet

EntityType Definition (XML)

```
<EntityType Name="A_SalesOrderType" sap:label="Sales Order">
  <Key>
    <PropertyRef Name="SalesOrder"/>
  </Key>
  <Property Name="SalesOrder" Type="Edm.String" MaxLength="10"
    sap:label="Sales Order" sap:filterable="true"/>
  <Property Name="SalesOrderType" Type="Edm.String" MaxLength="4"
    sap:label="Order Type"/>
  <Property Name="CreationDate" Type="Edm.DateTime"
    sap:label="Created On" sap:filterable="true" sap:sortable="true"/>
  <Property Name="TotalNetAmount" Type="Edm.Decimal" Precision="16" Scale="3"
    sap:label="Net Value" sap:filterable="false"/>
  <NavigationProperty Name="to_Item"
    Relationship="API_SALES_ORDER_SRV.assoc_SalesOrder_Item"
    FromRole="FromRole_assoc" ToRole="ToRole_assoc"/>
</EntityType>
```

SAP Annotations to Watch

Annotation	Meaning
sap:filterable="false"	Cannot use this property in \$filter
sap:sortable="false"	Cannot use this property in \$orderby

Edit with  Lovable

OData Guide

sap:creatable= false

Cannot set this property during POST

sap:updatable="false"

Cannot modify this property (read-only)

sap:required-in-filter="true"

Must include this property in \$filter



Always Check Metadata First

Before constructing queries, review the \$metadata to understand field restrictions. Attempting to filter on a non-filterable field will return an error, not an empty result.



Query Options Cheat Sheet

Query options let you filter, sort, page, and shape the data returned by OData services. All options start with \$.

\$filter

Restricts results to entities matching a condition.

```
$filter=SalesOrderType eq 'OR' and TotalNetAmount gt 1000
```

Comparison Operators

Operator	Meaning	Example
eq	Equal	Status eq 'A'
ne	Not equal	Status ne 'X'
gt	Greater than	Amount gt 100
ge	Greater than or equal	Amount ge 100
lt	Less than	Quantity lt 50
le	Less than or equal	Quantity le 50

Logical Operators

Edit with  Lovable

OData Guide

```
$filter=not (Status eq 'Cancelled')
```

DateTime Syntax

OData v2 uses `datetime'...'` format:

```
$filter=CreationDate ge datetime'2024-01-01T00:00:00'
```

```
$filter=CreationDate le datetime'2024-12-31T23:59:59'
```

\$select

Returns only specified properties, reducing payload size.

```
$select=SalesOrder, CustomerID, NetValue, CreationDate
```

\$orderby

Sorts results. Use `asc` or `desc`.

```
$orderby=CreationDate desc, SalesOrder asc
```

\$top & \$skip

Pagination controls. `$top` limits results; `$skip` offsets the starting point.

```
$top=50 # First 50 records
```

```
$top=50&$skip=50 # Records 51-100
```

```
$top=50&$skip=100 # Records 101-150
```

\$count / \$inlinecount

Get the total count of matching entities.

```
# Just the count (no data)
```

```
/A_SalesOrder/$count
```

OData Guide

\$format

Specify response format. Default is usually Atom/XML in OData v2.

```
$format=json
```

Filtering Examples

Effective filtering is crucial for performance. Here are common patterns for SAP OData services.

Date Filtering

Orders created in 2024

```
$filter=CreationDate ge datetime'2024-01-01T00:00:00'  
and CreationDate le datetime'2024-12-31T23:59:59'
```

Orders from the last 30 days (calculated client-side)

```
$filter=CreationDate ge datetime'2024-12-27T00:00:00'
```

String Filtering

Exact match

```
$filter=CustomerID eq 'CUST001'
```

Starts with (using substringof or startswith)

```
$filter=startswith(CustomerName, 'ACME')  
$filter=substringof('Corp', CustomerName)
```

Multiple Conditions

Edit with  Lovable

OData Guide

```
and TotalNetAmount gt 10000  
and CreationDate ge datetime'2024-01-01T00:00:00'
```

OR conditions

```
$filter=Status eq 'Open' or Status eq 'InProgress'
```

Numeric Filtering

Range filter

```
$filter=Quantity ge 10 and Quantity le 100
```



Non-Filterable Fields

Some fields cannot be used in filters. Common examples include:

- Calculated/aggregated fields (e.g., TotalNetAmount on some entities)
- Fields marked with `sap:filterable="false"`
- Navigation properties (filter on the target entity instead)

Always check `$metadata` before assuming a field is filterable.



Performance Tip

Use key fields and indexed properties in filters for best performance. Avoid using `substringof` on non-indexed fields as it may result in full table scans.



Selecting Columns

The `$select` option limits which properties are returned. This is a key optimization technique.

Benefits of \$select

- **Reduced payload:** Smaller response sizes mean faster transfers.
- **Faster processing:** The backend may skip expensive calculations for unused fields.

Edit with Lovable

OData Guide

BASIC \$select

```
$select=SalesOrder,CustomerID,CreationDate,TotalNetAmount
```

Combining with \$expand

```
$select=SalesOrder,CustomerID&$expand=to_Item($select=SalesOrderItem,Material,N
```



\$select Restrictions on Analytical Entities

Some SAP entities (particularly analytical/aggregate views) may not support \$select or may require specific field combinations. Common behaviors:

- All fields returned regardless of \$select
- Error if \$select is used at all
- Mandatory fields must always be included

The error CX_SADL_GW_ANALYT_GENERATED_ID often indicates \$select conflicts with analytical entity requirements.



Tip

When \$select doesn't work, try fetching all fields and filtering client-side. It's not ideal, but it's a practical workaround for restrictive entities.

↕ Sorting & Paging

Proper sorting and paging are essential for reliable data retrieval. Without them, you risk duplicates, missed records, and inconsistent results.

Why Stable Sorting is Critical

Without a stable sort, the server may return records in arbitrary order. When paging:

- ✗ Records can appear on multiple pages (duplicates)
- ✗ Records can be skipped entirely (gaps)
- ✗ Results are not reproducible across requests

Edit with Lovable

OData Guide

Recommended Pattern

```
# Sort by business field + unique ID for deterministic ordering
$orderby=CreationDate desc,SalesOrder asc&$top=100&$skip=0
```

```
# Page 2
$orderby=CreationDate desc,SalesOrder asc&$top=100&$skip=100
```

```
# Page 3
$orderby=CreationDate desc,SalesOrder asc&$top=100&$skip=200
```



Without Stable Sort

```
# ❌ BAD: Non-deterministic, may cause duplicates/gaps
$top=100&$skip=0
```

```
# ❌ RISKY: CreationDate alone may have duplicates
$orderby=CreationDate desc&$top=100&$skip=0
```

```
# ✅ GOOD: Include unique key for stability
$orderby=CreationDate desc,SalesOrder asc&$top=100
```

Server-Driven Paging

When the server enforces a page size limit, it returns a continuation token:

Response with __next link

```
{
  "d": {
    "results": [...],
    "__next": "https://host/sap/opu/odata/sap/SERVICE/EntitySet?$skiptoken='ABC'"
  }
}
```



Always check for and follow __next OR \$skiptoken :

Edit with  Lovable

OData Guide

```
let results = [];  
let nextUrl = initialUrl;  
  
while (nextUrl) {  
  const response = await fetch(nextUrl);  
  const data = await response.json();  
  results.push(...data.d.results);  
  nextUrl = data.d.__next || null;  
}  
  
return results;  
}
```



Best Practice

Even when using server-driven paging (\$skiptoken), always include \$orderby to ensure consistent ordering across pages. The skiptoken handles the offset, but order stability still depends on your sort clause.

Counting

Two approaches exist for counting records: the `/$count` path segment and the `$inlinecount` query option.

`/count` (Count Only)

Returns just the count as a plain integer—no data payload.

Get count only

```
GET /sap/opu/odata/sap/API_SALES_ORDER_SRV/A_SalesOrder/count
```

With filter

```
GET /sap/opu/odata/sap/API_SALES_ORDER_SRV/A_SalesOrder/count?$filter=Status eq 'A'
```

```
# Response: 1247
```



Edit with  Lovable

OData Guide

Data + inline count

```
GET /A_SalesOrder?$inlinecount=allpages&$top=50&$filter=Status eq 'Open'
```

Response includes:

```
{
  "d": {
    "__count": "1247",
    "results": [...]
  }
}
```

Performance Considerations

Method	Use Case	Performance
/\$count	When you only need the count	Fastest (no data serialization)
\$inlinecount	When you need data + count together	Adds overhead to each request



Count Can Be Expensive

For large datasets, counting may require scanning many records. SAP may:

- Return an estimated count instead of exact
- Time out on very large entity sets
- Restrict counting on certain analytical entities

Consider caching counts or using estimated values for UI pagination when exact counts aren't critical.



Tip

For pagination UIs, fetch the count once (or infrequently) and reuse it across page navigations rather than requesting \$inlinecount on every page.



Best Practices

Edit with Lovable

OData Guide

✔ Minimize Payload Size

Always use \$select to request only needed fields. Use \$filter to limit results server-side.

✘ Avoid

```
/A_SalesOrder
```

✔ Prefer

```
/A_SalesOrder?$select=SalesOrder,Status&$filter=...
```

✔ Always Include Stable Sort for Paging

Include a unique field in \$orderby to prevent duplicates and gaps across pages.

```
$orderby=CreationDate desc,SalesOrder asc&$top=100
```

✔ Prefer Server-Driven Paging

When available, use \$skiptoken from __next links instead of \$skip. It's more efficient and handles edge cases better.

```
// Check response for continuation
if (response.d.__next) {
  nextPageUrl = response.d.__next;
}
```

✔ Handle Metadata Restrictions

Before building queries, check \$metadata for field capabilities (filterable, sortable, etc.).

```
GET /sap/opu/odata/sap/API_SALES_ORDER_SRV/$metadata
```

✔ Monitor Error Logs

Edit with  Lovable

OData Guide

- Shows which backend component failed
- Helpful for debugging 500 errors that lack detail in responses

✔ Request JSON Format

JSON is more compact and easier to parse than Atom/XML.

Via query option

\$format=json

Or via Accept header

Accept: application/json

Troubleshooting

Common SAP OData errors and how to resolve them. When troubleshooting, always check `/IWFND/ERROR_LOG` in SAP for detailed server-side information.

`CX_SADL_GW_ANALYT_GENERATED_ID` Analytical Entity \$select Conflict

This error occurs when using \$select on analytical views that require specific field combinations or don't support projection.

Workaround: Remove \$select and fetch all fields, then filter client-side. Alternatively, check if the entity supports a specific set of mandatory fields that must be included together.

`CX_SDATA_NOT_FILTERABLE` Property Not Filterable

The field you're trying to filter on is marked as `sap:filterable='false'` in the metadata.

Workaround: Remove the problematic filter. Consider filtering on a related key field or fetching more data and filtering client-side.

Edit with  Lovable

OData Guide

Workaround: Choose a different field for sorting, ideally a key field that's always sortable.



403 Forbidden / Authorization Error **Missing Permissions**

Your user doesn't have the required authorizations for the requested entity or operation.

Workaround: Verify the user has appropriate SAP roles. Check transaction SU53 for authorization analysis. For technical users, ensure proper service assignment in PFCG.



504 Gateway Timeout **Request Timeout**

The request took too long. Common with large result sets or complex filters on non-indexed fields.

Workaround: Add more restrictive filters. Use \$top to limit results. Consider server-driven paging. Check if the entity has required filter fields.



400 Bad Request - Invalid Date **DateTime Format Error**

The datetime value isn't in the expected OData v2 format.

Workaround: Use datetime'YYYY-MM-DDTHH:MM:SS' format. Ensure proper URL encoding. Example: datetime'2024-01-15T00:00:00'



CSRF Token Required **Missing CSRF Token**

State-changing operations (POST, PUT, DELETE) require a valid CSRF token.

Workaround: First send a GET request with header 'x-csrf-token: Fetch', extract the token from response headers, then include it in your write request as 'x-csrf-token: <token>'.



__next Missing / Incomplete Data **Server Paging Limit Hit**

The server returned fewer records than expected. You've hit the server-enforced page limit.

OData Guide

Debugging Checklist

1. Check `/IWFND/ERROR_LOG` for server-side details
2. Verify `$metadata` for field restrictions (filterable, sortable, etc.)
3. Test the query in a browser or Postman with `$format=json`
4. Simplify the query—remove options one by one to isolate the issue
5. Check user authorizations in SU53
6. Verify the service is active in `/IWFND/MAINT_SERVICE`



Build Your Query

Interactively construct an OData URL

Base URL (Service Root)

`https://my-sap-system.com/sap/opu/odata/sap/API_SALES_ORDER_SRV`

Entity Set

SalesOrderSet



\$filter

+ Add Filter

No filters added

\$select (comma-separated)

`SalesOrder, CustomerID, NetValue`

\$orderby

`CreationDate desc`

\$top

`100`

Edit with  Lovable

OData Guide

Generated URL

`https://my-sap-system.com/sap/opu/odata/sap/API_SALES_ORDER_SRV/SalesOrderSet`



Frequently Asked Questions

What's the difference between OData v2 and v4 in SAP? 

Why am I getting 'Property cannot be filtered' errors? 

How do I handle authentication for OData calls? 

What's the maximum number of records I can retrieve? 

How do I call OData functions or actions? 

Can I update multiple records in a single request? 

Yes, using OData batch requests (\$batch). You can bundle multiple GET, POST, PATCH, or DELETE operations into a single HTTP POST to the \$batch endpoint. SAP supports both atomic (all-or-nothing) and non-atomic batch processing. Batch requests use multipart/mixed content type with specific formatting requirements.

How do I handle deep inserts (creating related entities)? 

What does the CSRF token error mean? 

OData Guide

A comprehensive reference for SAP OData endpoints, query options, and best practices.

Resources

[OData.org Documentation](#) 

[SAP Gateway Documentation](#) 

[SAP Business Accelerator Hub](#) 

Quick Links

[Query Options Reference](#)

[Query Builder Tool](#)

[Troubleshooting Guide](#)

Disclaimer: This guide is provided for informational purposes only. SAP, S/4HANA, and related marks are trademarks of SAP SE. This is an independent resource and is not affiliated with, endorsed by, or sponsored by SAP SE. Always refer to official SAP documentation for authoritative information.